TinyIDS: compression and quantization (RQ2)

Tools, steps, and the first test (`quant1`)

07 November 2026. Script: `model/compress.py`. Results: `model/results_quant1/quant_results.csv`, models: `model/models_quant1/`.

Contents

1 Why this step exists **2**

2 The tools **2**

3 The steps, per model **2**

4 First test results (`quant1`) **4**

5 A problem found and fixed: INT8 dynamic and big batches **6**

6 What the first test shows **6**

7 Next steps **7**

1 Why this step exists

The proposal (Methodology step 4) requires shrinking the trained models before they go to the Raspberry Pi:

> *"Prune tree depth and estimator count; convert the MLP/CNN to TensorFlow Lite or ONNX with INT8 quantization. On the Pi, measure model size (KB), peak RAM (MB), latency per flow (ms), throughput (flows/s) and energy per 1,000 inferences."*

and RQ2 (Research Question 2) asks: *which compressed model (quantized tree ensemble, pruned MLP, small 1D-CNN) gives the best accuracy per millisecond and per megabyte on the Pi?* Run 2's models were 7–54 MB, far above the target of < 1 MB.

Terms. Pruning = removing the parts of a model that matter least (fewer / smaller trees; weak neural-network connections). **Quantization** = storing numbers with fewer bits, e.g. 32-bit decimals $\rightarrow$ **INT8** (8-bit integers): about 4× smaller. **ONNX** = Open Neural Network Exchange, a universal model file format; **ONNX Runtime** runs ONNX files fast on almost any device, including the Pi. **MLP** = Multi-Layer Perceptron; **1D-CNN** = one-dimensional Convolutional Neural Network; **LightGBM** = Light Gradient Boosting Machine; **RF** = Random Forest; **TFLite** = TensorFlow Lite.

2 The tools

Tool (version)	What it does here
`onnx (1.23)`	Reads / writes ONNX model files.
`onnxruntime (1.31)`	Runs ONNX models; its `quantization` module does the INT8 conversion. Has a Raspberry Pi (ARM) version, so PyTorch is not needed on the Pi.
`torch.onnx (PyTorch 2.15)`	Exports the trained MLP / 1D-CNN from PyTorch to ONNX.
`onnxmltools (1.16)`	Converts LightGBM models to ONNX.
`skl2onnx (1.20)`	Converts scikit-learn models (Random Forest) to ONNX.
`LightGBM max_bin`	LightGBM sorts every feature into at most this many "bins"; 255 bins = 8 bits. We use 63: coarser thresholds, smaller trees.

Installed with `pip install onnx onnxruntime onnxmltools skl2onnx` (through the college proxy). Possible later extras (not used yet): `treelite` (compiles trees to a small C library), `hummingbird-ml` (turns trees into tensor operations so they can be INT8-quantized too).

3 The steps, per model

Neural networks (MLP, 1D-CNN): real INT8 quantization

1. **Start**: the PyTorch model from run 2 (45 features, no `IAT`), 32-bit.
2. **Export to ONNX (fp32)**: `torch.onnx.export(...)`. Same model, new file format; the score must not change.
3. **INT8 dynamic**: `onnxruntime.quantization.quantize_dynamic(...)`. The *weights* become 8-bit integers; the in-between values are converted on the fly while the model runs. Needs no extra data.

4. **INT8 static**: `onnxruntime.quantization.quantize_static(...)`. Weights *and* in-between values are 8-bit. It first runs 1,000 **calibration** rows from `train.csv` through the model to learn the value ranges.

One fix was needed for the 1D-CNN: its "adaptive average pooling" layer (squeezes 45 values into 8) cannot be exported to ONNX when 45 is not a multiple of 8. It was replaced at export time by an **exactly equivalent fixed matrix multiplication**; the script checks on 2,000 rows that the outputs are identical before continuing.

Tree models (LightGBM, Random Forest): pruning + coarser numbers + ONNX

Trees have no standard "INT8 button". For them, a "quantized tree ensemble" means fewer trees, smaller trees and coarser numbers inside:

1. **Start**: the run-2 model (45 features). LightGBM: up to 1,000 rounds, 63 leaves; RF: 100 trees, depth 20.
2. **Pruned**: retrained (20% sample) with only the **top 15 features** from the run-2 SHAP ranking (RQ1 sweet spot), and
   - LightGBM: at most 150 rounds, **15 leaves** per tree, `max_bin` **63**;
   - Random Forest: **30 trees**, **max depth 12**.
3. **Pruned → ONNX**: converted with `onnxmltools` / `skl2onnx` and run with ONNX Runtime, the same runtime as the neural networks.

Top-15 features used by the pruned trees

`Protocol Type, Tot size, Header_Length, Magnitue, Max, Min, Tot sum, flow_duration, Number, syn_count, Rate, Variance, urg_count, ack_count, psh_flag_number`

What is measured for every version

- **File size (KB)**: target < 1 MB (1,024 KB).
- **Accuracy** and **macro-F1** on the full `test.csv` (1,176,851 rows), and the **F1 change** versus the original model.
- **Latency**: time to classify **one** window (average over 300 single calls), **1 CPU core** for the trees, like on the Pi.
- **Throughput**: windows per second when 100,000 windows are classified in one batch.

These are **laptop** numbers. The Pi numbers (plus peak RAM and energy with the USB power meter) come from the benchmark script on the Pi later.

4 First test results (quant1)

Starting point: the run-2 models (no IAT). Pruned trees retrained on the 20% sample (1,098,395 rows). Every version scored on the **full** test.csv (1,176,851 rows). Laptop CPU. Took 7 minutes.

Binary (attack or normal)

Model	Version	Feat.	Size (KB)	Macro-F1	Change	ms / window
LightGBM	original	45	6,667	0.9450	—	0.710
LightGBM	pruned	15	260	0.9173	−0.028	0.681
LightGBM	**pruned → ONNX**	15	**157**	0.9173	−0.028	**0.019**
RF	original	45	7,292	0.9322	—	3.303
RF	pruned	15	659	0.9086	−0.024	1.090
RF	pruned → ONNX	15	859	0.9086	−0.024	0.012
MLP	original (PyTorch)	45	60	0.9158	—	0.036
MLP	ONNX fp32	45	57	0.9158	0.000	0.010
MLP	**ONNX INT8 dynamic**	45	**18**	**0.9158**	**0.000**	0.013
MLP	ONNX INT8 static	45	19	0.9152	−0.001	0.012
1D-CNN	original (PyTorch)	45	83	0.8792	—	0.112
1D-CNN	ONNX fp32	45	80	0.8792	0.000	0.034
1D-CNN	ONNX INT8 dynamic	45	27	0.8794	+0.000	0.052
1D-CNN	ONNX INT8 static	45	28	0.8803	+0.001	0.025

8-class (which attack family)

Model	Version	Feat.	Size (KB)	Macro-F1	Change	ms / window
LightGBM	original	45	26,916	0.7049	—	1.659
LightGBM	pruned	15	2,061	0.6635	−0.041	0.852
LightGBM	**pruned → ONNX**	15	1,290	**0.6635**	−0.041	**0.019**
RF	original	45	39,464	0.6876	—	3.507
RF	pruned	15	2,273	0.6348	−0.053	1.144
RF	pruned → ONNX	15	4,012	0.6348	−0.053	0.015
MLP	original (PyTorch)	45	62	0.6386	—	0.035
MLP	ONNX fp32	45	58	0.6386	0.000	0.015
MLP	**ONNX INT8 dynamic**	45	**19**	**0.6397**	+0.001	0.013
MLP	ONNX INT8 static	45	19	0.5972	−0.042	0.017
1D-CNN	original (PyTorch)	45	84	0.6111	—	0.116
1D-CNN	ONNX fp32	45	82	0.6111	0.000	0.041
1D-CNN	ONNX INT8 dynamic	45	28	0.6115	+0.000	0.074
1D-CNN	ONNX INT8 static	45	28	0.5702	−0.041	0.034

"ms / window" = time to classify one window on the laptop (trees: 1 CPU core). Throughput per batch is in `quant_results.csv`.

5 A problem found and fixed: INT8 dynamic and big batches

In the first attempt, **INT8 dynamic collapsed**: binary MLP macro-F1 0.916 → **0.023**, 8-class MLP and CNN → 0.105, while INT8 static was fine (log kept as `compress_quant1_batch-eval.log`).

Cause: a dynamic INT8 model chooses its 8-bit scale *from the whole input it is given.* The test set was given as one batch of 1.18 million rows. A handful of rows have extreme values after preprocessing (e.g. `Drate` is almost always 0, so its rare non-zero values become enormous after standardisation). Those few rows stretched the scale so much that all other rows were squashed into one or two integer values.

Fix: the live IDS classifies **each window on its own**, so dynamic models are now scored one window at a time, exactly as they will be used. Result: **no loss at all** (binary MLP 0.9158 → 0.9158).

Lessons: (1) never feed a dynamic INT8 model a big mixed batch; (2) later, clip extreme standardised values (e.g. to ±10) in the preprocessing, which makes both INT8 versions more robust.

6 What the first test shows

1. **Neural networks: INT8 quantization works almost for free.** MLP 60 KB → **18 KB** (3.3× smaller) with **no F1 loss** (dynamic). Static INT8 is just as small and the fastest, but loses 0.04 macro-F1 on 8-class (its fixed calibration ranges clip some rows).
2. **ONNX alone makes everything much faster.** Trees: 0.7–3.5 ms → **0.01–0.02 ms** per window (35–230×), because ONNX Runtime skips Python overhead. Neural networks: 2–4× faster.
3. **Trees: pruning shrinks them a lot**, e.g. binary LightGBM 6.7 MB → **157 KB** (42×), 8-class 26.9 MB → 1.3 MB (21×).
4. **But pruned trees lose accuracy**: binary −0.024 to −0.028, 8-class −0.041 to −0.053 macro-F1. Part of this is using only 15 features (RQ1 showed 15 features cost about 0.03 on their own), part is the smaller trees.
5. **ONNX made Random Forest files bigger** (binary 659 → 859 KB; 8-class 2.3 → 4.0 MB): `joblib` compresses its file, ONNX does not. LightGBM got smaller in ONNX.

Against the targets (laptop numbers)

Target	Best result so far	Status
Model size < 1 MB	Binary: all compressed versions (18–859 KB). 8-class: MLP / CNN INT8 (19–28 KB), LightGBM ONNX 1,290 KB	binary met; 8-class: LightGBM slightly over (1.3 MB), RF well over (4 MB)
Latency < 5 ms per window	every ONNX version ≤ 0.07 ms on the laptop	met (re-check on the Pi, 3–5× slower)
Throughput ≥ 500 / s	13,000 – 4,000,000 / s	met
F1 loss small	NN INT8 dynamic: 0.000; trees: 0.024–0.053	NN yes; trees need tuning

Best candidates for the IDS right now

- **Binary**: LightGBM pruned → ONNX (157 KB, F1 0.917) or MLP INT8 (18 KB, F1 0.916): same F1, the MLP is 9× smaller, but LightGBM works directly with TreeSHAP for the expla-

nations.

- **8-class**: LightGBM pruned → ONNX (1.3 MB, F1 0.664) is the most accurate compressed model; MLP INT8 (19 KB, F1 0.640) the smallest.

7 Next steps

TO DO NEXT for the model (decided 07 Nov 2026; parked while the testbed is built)
Tune the pruned trees so compression costs less accuracy:

- try **25–30 features** instead of 15 (RQ1: the curve flattens after 15, but 30 was within 0.02 for LightGBM);
- try **31 leaves per tree** instead of 15;
- for 8-class, use **fewer rounds** (every round adds 8 trees) and/or fewer bins;
- **goal**: macro-F1 loss **below 0.02** versus the original, and **every** model **under 1 MB**.

Command to re-run after changing the settings in `compress.py`: `python compress.py --run-name quant2`

1. (see box) Tune the pruned trees.
2. Use the **7-class** family model (DDoS + DoS merged), as planned for the IDS.
3. Clip extreme preprocessed values for the neural networks.
4. Run the same comparison **on the Pi** with peak RAM and energy (`rq2_benchmark.py`) → final RQ2 table.